

Govind Charpe

github.com/gcharpe1604

govindcharpe8@gmail.com

LFX MENTORSHIP · SUMMER 2026



HARBOR
HARBOR



CNCF

Harbor CLI · goharbor/harbor-cli

TO

Vadim Bauer, Prasanth Baskar, and Orlin Vasilev

Mentors — Harbor CLI · CNCF / Linux Foundation LFX Mentorship

Application — *Harbor CLI*, LFX Mentorship Summer 2026

I started contributing to Harbor CLI before applying to LFX Mentorship because I wanted to understand how the project was actually maintained, reviewed, and architected in practice. Over time, those contributions expanded beyond isolated fixes into broader reliability and workflow consistency improvements across the Harbor ecosystem.

Contribution context

Over the past several weeks of contributing within the Harbor ecosystem, I have opened seven pull requests across Harbor ecosystem projects (including [goharbor/harbor-cli](#) and [container-registry/harbor-satellite](#)), spanning prompt error handling, configuration reliability, cross-file system robustness, security ergonomics, and test suite stability.

PR #849 was merged by Prasanth Baskar; #855 was approved by Patrick Eschenbach (qcserestipy); #897 was approved by NucleoFusion. Four further PRs (including my recent fixes for Harbor Satellite's Zot config management and Windows/container testing) are currently under active review. Several of these PRs evolved significantly through maintainer review iterations, especially around workflow reliability, failure propagation, and subsystem consistency.

Harbor ecosystem contributions

7	1	2	4
PRS OPENED	MERGED	APPROVED	IN REVIEW

Harbor CLI contributions

PR	CHANGE & IMPACT	AREA	STATUS
#849	Prompt error handling refactor Replaced <code>log.Fatal()</code> in <code>pkg/prompt/prompt.go</code> with <code>(value, error)</code> returns, restoring caller-controlled failure handling and separating input collection from process lifecycle. <i>Merged by Prasanth Baskar · Approved by Patrick Eschenbach (qcserestipy)</i>	Prompt subsystem	Merged
#855	Login <code>--password</code> flag security guidance Updated the <code>--password</code> flag description in <code>cmd/harbor/root/login.go</code> to warn about shell history exposure and steer users toward the safer <code>--password-stdin</code> path while following the same convention as <code>docker login</code> . <i>Approved by Patrick Eschenbach (qcserestipy)</i>	Login · CLI UX	Approved
#897	Regex precompilation in <code>pkg/utils</code> Moved inline regex literals in <code>pkg/utils/helper.go</code> to package-scope <code>var</code> blocks, preventing recompilation on every validation call and aligning with standard Go best practices. <i>Approved by NucleoFusion</i>	Utils · Code quality	Approved
#930	Silent success and context-switch error propagation fix Added nil checks in <code>configurations/apply.go</code> to reject empty configs instead of exiting cleanly, and fixed context-switch failures that printed an error but returned exit 0, breaking automation pipelines.	Config · CLI reliability	In Review
#932	Replication and retention workflow reliability refactor Refactored replication and tag-retention interactive workflows to replace abrupt <code>log.Fatal()</code> / <code>os.Exit()</code> termination with idiomatic Cobra error propagation. Fixed destructive abort behaviour in retention deletion flows, removed API panic vectors, unified Bubble Tea cancellation handling, and simplified prompt concurrency through synchronous error propagation.	Replication · Operational reliability	In Review

PR	CHANGE & IMPACT	AREA	STATUS
#456	Cross-device rename error (EXDEV) fix in Zot Prevented EXDEV rename failures in the Zot configuration manager by creating temporary configuration files directly within the target directory instead of the system temp directory.	Zot integration	In Review
#454	Windows & container test suite robustness Resolved test suite failures on Windows and minimal container environments by dynamically skipping platform-specific boot/machine ID retrieval and adjusting file permission validation for Windows systems.	Satellite Testing	In Review

Engineering approach

Most of my contributions so far have focused on the same general problem: places where Harbor CLI behaves correctly during normal usage, but becomes unreliable under failure conditions, automation workflows, or long-term maintenance pressure. PRs #849 and #932 are the clearest examples. What initially started as relatively small error-propagation fixes eventually uncovered broader reliability problems around cancellation handling, destructive defaults, panic vectors, and subsystem-level failure consistency. In several cases, the deeper issue was not the immediate bug itself, but that lower-level helpers and interactive workflows were making process-level failure decisions that should instead belong to callers and command orchestration layers. Returning errors instead restores composability and predictable failure handling while also simplifying several unnecessarily complex prompt workflows.

The same thinking applies to #930: an empty config that exits cleanly is worse than one that fails loudly, because silent success is dangerous when CLI tools should fail explicitly when something goes wrong. Fixing that is a maintainability change, not just a bug fix.

Implementation direction under mentorship

The proposed Bubbletea v2 migration and authentication modernization work align closely with the areas of Harbor CLI I have already spent the most time understanding: interactive workflow reliability, subsystem consistency, and predictable operational behaviour across both user-facing and automation-driven command flows.

PHASE	FOCUS	SUBSYSTEM	GOAL
01	Bubbletea v2 migration foundation Standardize asynchronous TUI behaviour and prepare migration-safe abstractions across existing interactive flows.	TUI Core	Foundation
02	Unified <code>loadingtablelist</code> abstraction Consolidate fragmented list/table implementations into reusable loading, pagination, and selection components.	UI Abstractions	Refactor
03	OAuth2/OIDC device-flow integration Introduce secure token-based authentication for interactive users and CI/CD automation workflows.	Authentication	Security
04	Reliability and operational consistency Improve testing coverage, predictable CLI behaviour, subsystem maintainability, and automation reliability.	Architecture	Stability

The first implementation focus would be stabilizing the current TUI fragmentation before introducing larger refactors. The Bubbletea v2 migration is not just a dependency upgrade, but an opportunity to standardize asynchronous interaction behaviour across Harbor CLI's existing `tablelist`, `selection`, `multiselect`, and related flows using more consistent state-update patterns through `tea.Init()`, `tea.Tick()`, and reusable model abstractions for future TUI development.

A major follow-up direction would involve consolidating the current table and list implementations into a unified `loadingtablelist` abstraction with shared pagination, asynchronous loading, error-state propagation, and reusable navigation behaviour across interactive command flows. In parallel, the OAuth2/OIDC device authorization work proposed for `harbor login --sso` introduces a more secure authentication model for both developers and CI/CD environments through provider-agnostic OIDC integration, secure device authorization workflows, encrypted credential storage, and reliable non-interactive authentication support for automation environments.

Reliability and maintainability would remain a consistent focus throughout both projects. My earlier contributions around caller-controlled error propagation, prompt consistency, and predictable CLI failure handling already touch related concerns at a smaller scale. Under mentorship, I would continue extending those principles more broadly across interactive flows, subsystem-level error handling, testing infrastructure, and long-term operational consistency inside Harbor CLI.

Working directly with Vadim Bauer, Prasanth Baskar, and Orlin Vasilev would provide the architectural context that cannot be gained from reading the codebase alone: understanding migration trade-offs, subsystem boundaries, and where engineering effort has the highest long-term leverage inside Harbor CLI.

How I work

I respond to reviewer feedback quickly and update PRs incrementally. Across the contributions so far, iteration has been the norm: reading feedback carefully, making targeted changes, and keeping the PR moving without requiring follow-up from maintainers. I try to make reviewing my work as low-friction as possible: clear commit messages, scoped changes, and PR descriptions that explain the reasoning behind a fix, not just what changed.

Availability

20–25 hours per week through the summer term, with flexibility to do more during active development phases. This mentorship is my primary focus for the summer.

My full contribution history is visible on GitHub. I would be happy to discuss any of the above PRs in more detail, including the reasoning behind them, the review cycles, or the trade-offs considered.

Govind Charpe

github.com/gcharpe1604 govindcharpe8@gmail.com